FPGA AI Suite Full Flow Guidance for Agilex 7 FPGA F-Series Development Kit

Target board: Agilex 7 FPGA F-Series Development Kit, Production 1, 2x F-Tile

Likely kit family: Agilex 7 FPGA F-Series 2x F-Tile, AGF023-class platform

Workstation: Intel Xeon W5-3433, Windows 11 Pro + Ubuntu dual boot, 256 GB RAM, NVIDIA RTX 5000 Ada

Research model: pretrained BERT model for MRPC paraphrase detection

Purpose: installation, board setup, FPGA AI Suite deployment path, and support items to request from Altera/Intel technical team

1. Executive summary

The workstation is suitable for FPGA AI Suite development and Quartus Prime Pro compilation. The software flow for the BERT model is already well aligned with FPGA AI Suite: PyTorch model training, ONNX export, OpenVINO IR conversion, DLA Compiler analysis, and heterogeneous CPU-FPGA evaluation.

The main technical caveat is board support. In the current FPGA AI Suite 2026.1.1 design-example matrix, the Agilex 7 F-Series 2x F-Tile development kit is not listed as an out-of-box FPGA AI Suite target. The listed Agilex 7 AI Suite examples target boards such as Agilex 7 I-Series Development Kit, DE10-Agilex, and N6001. Therefore, using the F-Series kit may require a board-specific port, BSP/OFS support package, or a custom Quartus integration of the FPGA AI Suite IP.

Recommended decision path

For this university engagement, the most concrete technical path is:

1. Install and validate the software toolchain first: Quartus Prime Pro, FPGA AI Suite, OpenVINO, license, and JTAG.

2. Verify the Agilex 7 F-Series kit independently using the board vendor’s Golden Top, Board Test System, memory, PCIe, and JTAG examples.

3. Continue using FPGA AI Suite DLA Compiler for BERT model analysis, partitioning, and performance/resource estimation.

4. Ask Altera support to confirm whether an official FPGA AI Suite board support package or design identifier exists for the Agilex 7 F-Series 2x F-Tile kit.

5. If no official design exists, treat the FPGA hardware stage as a board-porting task: adapt an Agilex 7 AI Suite reference design to the F-Series board, including PCIe/OFS, DDR, clocks, resets, pin constraints, and runtime plugin integration.

What can be committed immediately

The following activities can proceed immediately on the workstation:

- Toolchain installation and validation.
- Model conversion and FPGA AI Suite compiler analysis.
- Quartus/JTAG board bring-up using board-provided examples.
- Collection of compiler estimates for BERT: area, estimated throughput, unsupported layers, and CPU/FPGA partitioning.

The following activities require board-specific confirmation or porting:

- Final FPGA AI Suite hardware bitstream for the Agilex 7 F-Series kit.
- PCIe/OFS runtime flow on the exact F-Series board.
- End-to-end BERT execution on that FPGA board.

2. Recommended software versions

Use one consistent release set. Do not mix versions from older documents unless directed by Altera support.

Component	Recommended version
FPGA AI Suite	2026.1.1
Quartus Prime Pro	26.1
OpenVINO Toolkit	2025.4
Open Model Zoo tools	2024.6 if using ‘omz_downloader‘ / ‘omz_converter‘
Python	3.12
Ubuntu native support	22.04 LTS or 24.04 LTS
Windows support	Docker-based FPGA AI Suite flow

For Ubuntu 26.04 or other unsupported host versions, use the FPGA AI Suite Docker image instead of native installation.

3. Important board-support clarification

Officially listed FPGA AI Suite design examples

The FPGA AI Suite 2026.1.1 design-example documentation lists targets such as:

- Agilex 5 FPGA E-Series 065B Modular Development Kit
- Agilex 7 FPGA I-Series Development Kit ES2
- Terasic DE10-Agilex Development Board
- FPGA SmartNIC N6001-PL Platform
- Agilex 7 I-Series Transceiver-SoC Development Kit
- Arria 10 SX SoC FPGA Development Kit

Board in this request

The requested board is:

```
Agilex 7 FPGA F-Series Development Kit, Production 1, 2x F-Tile
```

This board is a high-speed transceiver and PCIe/CXL-capable F-Series platform. It is not the same as the Agilex 7 I-Series board called out by the FPGA AI Suite examples.

Practical implication

The model can be compiled and analyzed with FPGA AI Suite using an Agilex 7 architecture file, but producing a working FPGA implementation on this exact F-Series board may require one of the following:

1. An official Altera FPGA AI Suite example/BSP for this exact F-Series kit.
2. Porting an existing Agilex 7 FPGA AI Suite example design to the F-Series kit.
3. Custom integration of FPGA AI Suite IP into a Quartus project for this board.

4. A board-specific PCIe/OFS, DDR, clocking, reset, and pin assignment adaptation.

This should be explicitly confirmed with Altera technical support before committing to a deployment schedule.

4. Requested support from Altera/Intel

Ask Altera support to confirm/provide:

1. Whether FPGA AI Suite 2026.1.1 officially supports the Agilex 7 F-Series 2x F-Tile kit.
2. The correct board design identifier, if one exists.
3. The recommended architecture file for this device and board.
4. Any required OFS/BSP/reference design package for the F-Series kit.
5. Whether a PCIe-attached AI Suite design exists for this board.
6. Required kernel drivers, PCIe drivers, and runtime plugin configuration.
7. Required license features for FPGA AI Suite/CoreDLA and Quartus Prime Pro.
8. Recommended path for integrating a BERT/OpenVINO model into the FPGA implementation.
9. Known limitations for transformer/BERT-style models in FPGA AI Suite.
10. Recommended method to measure latency, throughput, FPGA IP throughput, and end-to-end system throughput.

5. Installation flow

5.1 Windows 11 path

For Windows, use Docker. Native Windows is not the preferred path for FPGA AI Suite development.

```
docker pull alterafpga/fpgaaisuite:2026.1.1-quartus
```

Create a workspace:

```
mkdir C:\fpga-ai-work
```

Run container:

```
docker run -it --name fpga-ai-suite-2026 -v C:\fpga-ai-work:/workspace alteraf
  pga/fpgaaisuite:2026.1.1-quartus
```

What this does: starts FPGA AI Suite in a Linux container with OpenVINO, FPGA AI Suite tools, and Quartus tools.

Why required: it avoids Windows dependency/version mismatch and gives a reproducible FPGA AI Suite environment.

5.2 Linux path

For Ubuntu 22.04/24.04, native or Docker is possible. For newer unsupported Ubuntu releases, Docker is recommended.

```
docker pull alterafpga/fpgaaisuite:2026.1.1-quartus
```

Recommended startup with workspace, JTAG, and license mount:

```
docker run -it \
  --name fpga-ai-suite-2026 \
  --network host \
  --privileged \
  -v /dev/bus/usb:/dev/bus/usb \
  -v ~/fpga-ai-work:/workspace \
  -v /path/to/license.dat:/licenses/altera_license.dat:ro \
  -e LM_LICENSE_FILE=/licenses/altera_license.dat \
  -e ALTERAD_LICENSE_FILE=/licenses/altera_license.dat \
  alterafpga/fpgaaisuite:2026.1.1-quartus
```

What this does: starts the full Quartus-enabled FPGA AI Suite image and exposes USB-JTAG, network host ID, workspace, and license file.

Why required: Quartus and FPGA AI Suite license checks often depend on host ID; --network host helps node-locked license validation in Docker.

6. Tool verification

Inside Docker:

```
python3 -c "import openvino; print(openvino.__version__)"
echo $COREDLA_ROOT
dla_compiler --version
quartus_pgm --version
jtagconfig
```

Expected:

```
OpenVINO 2025.4.x
FPGA AI Suite 2026.1.1
Quartus Programmer 26.1
```

If using a node-locked license, verify:

```
lmutil lmhostid
lmutil lmdiag -c /licenses/altera_license.dat <feature_id>
```

Success should indicate that the license is valid for the current node.

7. Initial board setup and verification

7.1 Quartus board-level checks

Before using FPGA AI Suite, verify the board with vendor-provided kit tools:

1. Install Quartus Prime Pro 26.1.
2. Install the Agilex 7 F-Series device support package.
3. Install board package/design examples if provided.
4. Verify JTAG visibility.
5. Run Board Test System or Golden Top design if available.
6. Verify DDR, PCIe, clocks, QSFP, and transceiver-related examples as applicable.

7.2 JTAG detection

```
jtagconfig
```

What this does: checks whether Quartus can detect the USB-JTAG cable and board devices.

Why required: programming and System Console debug cannot proceed without JTAG visibility.

7.3 Program a simple board test bitstream

```
quartus_pgm -c 1 -m jtag -o "p;<board_test_or_golden_top>.sof"
```

What this does: programs the FPGA over JTAG.

Why required: verifies the basic programming path before attempting FPGA AI Suite hardware integration.

8. Model preparation flow for BERT

The research flow already completed is correct:

1. Train or fine-tune BERT in PyTorch using MRPC.
2. Export PyTorch model to ONNX.
3. Verify ONNX inference output against PyTorch.
4. Convert ONNX to OpenVINO IR using OVC.
5. Generate FP32 and optionally FP16 IR.
6. Run FPGA AI Suite DLA Compiler against an Agilex 7 architecture file.
7. Review unsupported layers, CPU fallback, and FPGA partitioning.
8. Generate performance and resource estimates.
9. Evaluate in HETERO CPU-FPGA mode where applicable.

Example conversion:

```
ovc bert_mrpc.onnx --output_model bert_mrpc_fp32.xml
```

What this does: converts ONNX to OpenVINO IR.

Why required: FPGA AI Suite compiler consumes OpenVINO IR model representation.

9. DLA Compiler analysis

Example command:

```
dla_compiler \
  --march $COREDLA_ROOT/example_architectures/AGX7_Generic.arch \
  --network-file ./bert_mrpc_fp32.xml \
  --o $COREDLA_WORK/demo/BERT_AGX7_Generic.aot \
  --fanalyze-performance
```

What this does: compiles/analyzes the OpenVINO model for a selected FPGA AI Suite architecture.

Why required: it estimates performance, identifies FPGA/CPU partitions, and creates an AOT compiled model if possible.

Important outputs:

```
compiled_model_dir/<network>/reports/performance-report_0.txt
compiled_model_dir/<network>/model_analyzer_report.txt
unsupported layer messages
AOT compiled model
```

Area estimate:

```
dla_compiler --fanalyze-area --march $COREDLA_ROOT/example_architectures/AGX7_
  Generic.arch
```

What this does: estimates FPGA AI Suite IP resource usage for the selected architecture.

Why required: helps judge whether the selected architecture can fit the target FPGA device.

10. Moving from optimized model to FPGA implementation

Supported-board path

For officially supported FPGA AI Suite boards, use the appropriate dla_build_example_design.py design identifier.

Example pattern:

```
$COREDLA_ROOT/bin/dla_build_example_design.py build \
  --licensed \
  --output-dir <build_dir> \
  --num-instances 1 \
  --seed 1 \
  <design_example_identifier> \
  <architecture_file>
```

F-Series board path

For the Agilex 7 F-Series 2x F-Tile kit, first confirm whether Altera provides a board-specific design identifier. If not, the design must be ported.

Porting typically requires:

1. Quartus project for the F-Series kit.
2. Correct device selection and board pin assignments.
3. Correct clock/reset topology.
4. DDR4 memory controller integration.
5. PCIe/CXL/OFS subsystem integration if using host-attached inference.
6. FPGA AI Suite IP integration.
7. Runtime plugin and memory-map consistency.
8. Matching .arch file between compiler and hardware.
9. Timing closure in Quartus.
10. Board-specific programming and runtime validation.

11. Quartus compilation and programming file generation

Expected Quartus flow:

```
Analysis & Synthesis
Fitter / place-and-route
Timing Analyzer
Assembler
Programming file generation
```

Successful output:

```
.sof for JTAG programming
.jic or flash image if persistent boot is required
fit/timing/QoR reports
```

Example programming:

```
quartus_pgm -c 1 -m jtag -o "p;<generated_bitstream>.sof"
```

What this does: configures the FPGA with the compiled design.

Why required: the FPGA must contain the hardware accelerator design before runtime execution.

12. Runtime execution and verification

A runtime command generally needs:

```
model XML or compiled AOT model
plugin XML
architecture file
input data
FPGA device selection
runtime design-specific parameters
```

Example pattern:

```
./dla_benchmark \
  -b=1 \
  -m=<model.xml> \
  -d=HETERO:FPGA,CPU \
  -i=<input_images> \
  -niter=<iterations> \
  -plugins=<plugins.xml> \
  -arch_file=<architecture.arch> \
  -api=async \
  -perf_est \
  -dump_output
```

What this does: runs inference through OpenVINO and FPGA AI Suite runtime.

Why required: validates real deployment behavior and collects measured performance.

13. Measurement plan

Collect these metrics:

```
| Metric | Source |
|---|---|
| FPGA resource utilization | Quartus fit/QoR reports |
| ALMs, DSPs, RAM blocks, M20Ks | Quartus and DLA area reports |
| Estimated IP throughput | ‘dla_compiler --fanalyze-performance‘ |
| Measured IP throughput | ‘dla_benchmark -perf_est‘ output |
```

```
| System throughput | ‘dla_benchmark‘ output |
| Latency | ‘dla_benchmark‘ output |
| Clock frequency | runtime and timing reports |
| CPU fallback layers | compiler/model analyzer reports |
| Unsupported layers | DLA messages file |
```

For BERT/transformer workloads, carefully inspect unsupported operators and CPU fallback. Transformer models may not map fully to FPGA AI Suite IP, depending on operator coverage and architecture settings.

14. Recommended phased plan

Phase 1: Workstation and software bring-up

- Install Docker/Quartus image or native supported toolchain.
- Verify OpenVINO, FPGA AI Suite, Quartus, JTAG.
- Verify license.

Phase 2: Board bring-up

- Verify JTAG.
- Program vendor-provided Golden Top/BTS design.
- Run board memory/PCIe tests.

Phase 3: Model compiler validation

- Compile BERT OpenVINO IR with Agilex 7 architecture.
- Review unsupported layers and CPU fallback.
- Capture performance/resource estimates.

Phase 4: FPGA AI Suite design implementation

- Use official board-specific AI Suite design if available.
- If unavailable, port a reference design to the Agilex 7 F-Series kit.
- Integrate FPGA AI Suite IP, memory subsystem, PCIe/OFS/HPS path, and runtime plugin.

Phase 5: Hardware deployment

- Compile Quartus design.
- Generate ‘.sof‘/‘.jic‘.
- Program board.
- Run model through FPGA AI Suite runtime.
- Measure throughput, latency, and resource utilization.

15. Key risks and mitigations

```
| Risk | Mitigation |
|---|---|
| F-Series board not listed as AI Suite example target | Ask Altera for official BSP/reference design or porti
| BERT operators may not fully map to FPGA | Inspect compiler partitioning and unsupported layer reports |
| PCIe/OFS setup may be board-specific | Request F-Series PCIe/OFS package and driver instructions |
| License mismatch | Verify with ‘lmutil lmdiag‘ before long Quartus builds |
| Timing closure | Start with generic architecture, then optimize |
| End-to-end throughput lower than IP throughput | Separate host transfer overhead from FPGA IP throughput |
```

16. Suggested message to technical support

We request confirmation whether FPGA AI Suite 2026.1.1 provides an official design example, board support package, or OFS PCIe flow for the Agilex 7 FPGA F-Series Development Kit, Production 1, 2x F-Tile. If not, please provide recommended porting steps from the closest supported Agilex 7 FPGA AI Suite design example to this F-Series kit, including board-specific Quartus project setup, PCIe/OFS integration, DDR interface setup, runtime plugin configuration, and recommended architecture file for deploying an OpenVINO BERT model compiled with the FPGA AI Suite DLA Compiler.

17. Conclusion

The software/model preparation flow is ready through OpenVINO and FPGA AI Suite DLA Compiler. The workstation is suitable for Quartus and FPGA AI Suite development. The main open item is board-specific FPGA AI Suite hardware support for the Agilex 7 F-Series 2x F-Tile development kit. If Altera provides an official board-specific reference design, deployment can follow that path. Otherwise, FPGA implementation requires porting/integration of the FPGA AI Suite IP into a Quartus design for this exact board.

This document is strong enough to share as a technical planning and support-request document because it separates confirmed software flow from board-specific assumptions. It should not be presented as a guarantee of out-of-box FPGA AI Suite support for the F-Series kit unless Altera confirms that support separately.

18. Authorship and disclaimer

Prepared by Sahil Patni as a technical guidance note based on publicly available Altera documentation, FPGA AI Suite workflow analysis, and prior FPGA AI Suite bring-up experience.